

TECU Benchmark — Master Plan

Built around the 3-pipeline vision plus every wall we've hit or will hit. Ordered for maximum efficiency: solve foundations first; each fix unlocks multiple downstream walls.

TECU is the benchmark — the hardest real app we can find. If our agent works here, it works almost anywhere.

Design Philosophy — Precision Within Generous Bounds

This is not a low-latency chat agent. This is a precision-focused data-extraction + testing suite.

When we must choose between fast and correct, we choose correct — **but every operation has a hard ceiling** that triggers a graceful exit. No infinite retries. No unbounded runs. No "just one more try" loops.

Philosophy

Dimension	Approach
Primary target	Correct output on first run. Reruns are the real cost, not per-run time.
Retry strategy	Bounded: N attempts, then graceful-exit with partial save + clear failure reason.
Verification	Every LLM claim verified before commit. But verification itself is bounded — max K verify calls.
Uncertainty surfacing	"Unsure" is a valid response. Low-confidence outputs are flagged, never swept under the rug.
Determinism	Python deterministic where possible. LLM only where perception / judgment is genuinely needed.
Failure mode	Save after every element. Resume-from-N on restart. Never restart whole runs for single-element fail.
Cost posture	Phase 1 (now): lenient bounds — get it working. Phase 2 (later): tighten bounds once working.

Operational Guardrails — Tight Caps Aligned To Measured Baseline

Our actual measured costs: \$0.04-\$0.07 per page extract, \$0.02 for plan generation. Precision mode accepts **at most 2x current cost** — and we achieve it through smarter engineering (caching, compound tools, cheaper models for verification), not by throwing more LLM calls at the problem.

Scope	Soft Cap (warn)	Hard Cap (exit)	On Hard Cap Hit
Per-element extraction	2 LLM calls / 10 sec	3 calls / 20 sec / \$0.015	Mark element <code>needs_review: true</code> , save what we have, next element.

Scope	Soft Cap (warn)	Hard Cap (exit)	On Hard Cap Hit
Per-dropdown option pass	1 extract + 1 verify	2 cycles / \$0.02	Mark dropdown options: [] , confidence: 0.3 , move on.
Per-page extraction	\$0.15 / 5 min	\$0.30 / 12 min / 80 calls	Save partial KB, log remaining fields, exit cleanly with summary.
Per-test execution	2 LLM calls / 30 sec	3 calls / 90 sec / \$0.05	Mark test BLOCKED with reason, continue to next test.
Per-pipeline run	\$0.75 / 15 min	\$1.50 / 30 min	Save all completed work, halt cleanly with "BUDGET_EXCEEDED" status.
Post-action verification	1 verify, no retry	1 verify / \$0.005 extra	Accept claim with capture_confidence: 0.6 , flag for later review.
CoVe verification loop	1 verify + 1 revise	2 verifies total	Keep last verified version, add verify_loop_hit_cap: true .

Total expected full-run cost: \$0.20-\$0.40 for 6-page TECU extract, \$0.50-\$1.00 for full Execute. If we hit these caps, either we have a bug or the app is genuinely abnormal.

💡 How We Stay Cheap While Adding Precision

Precision doesn't mean "more LLM calls". It means engineering-first verification with LLM only where it genuinely helps.

Technique	How It Cuts Cost
Prefix caching	Same system prompt across sub-tasks → 90% cached on subsequent calls. We saw 97% cache hit today.
Compound tools	Python does deterministic multi-step work (click-wait-click-snapshot) — LLM sees only the decision point.
Haiku for verification	gpt-5.1 (\$1.25 / \$10) → haiku (\$1 / \$5) for verify passes. ~50% cheaper on the verify portion.
Batch verification	Verify 5 claims in one LLM call instead of 5 calls. ~80% savings on verify overhead.
Structural verification (free)	Python regex-checks catch 80% of hallucinations before any LLM verify is even needed.
Skip verify on deterministic data	Native <select> options from JS enumerate — no hallucination risk, no verify.
Cheap confidence heuristics	"Element has DOM id + label match + not disabled = confidence 0.95, skip verify." Rule-based first.
Context trimming per sub-task	Each LLM call gets ONLY the snapshot lines relevant to the question — not the whole tree.

We already applied these well to get to \$0.07/page. We extend the same discipline to precision: **add verification, but make it cheap through engineering.**

△ Graceful Exit Contract

Every orchestrator / sub-task that hits a hard cap MUST:

- 1. **Save partial state** to the KB / results file before returning
- 2. **Log a structured reason:** `{exit_reason: "budget_exceeded" | "time_exceeded" | "max_retries" | "unsure_limit", details: str}`
- 3. **Return a partial: true flag** so downstream callers know to handle incompleteness
- 4. **Never raise** unless the failure is non-recoverable (MCP server dead, filesystem error)

No pipeline run should ever hang, spin, or burn budget past the cap. If it does, that's a bug.

Cost Phases

Phase 1 (now — build precision within tight bounds): caps above enforced from day one. Target \$0.20-\$0.40 per full 6-page extract, \$0.50-\$1.00 per full Execute run. We design for these numbers using the techniques in the table above.

Phase 2 (post-TECU — compress further): once everything works, target another ~40% reduction:

- Haiku-only for all verification sub-tasks
- Cache snapshot embeddings across sub-tasks
- Skip CoVe on elements with ≥10 prior runs showing 100% accuracy
- Pre-compute per-app "known-safe" patterns so we skip verification on familiar fields

No unchecked spending. If a run exceeds the hard cap, that's a bug (loop, hallucination, runaway retry) — not a signal to raise the cap. Budget guardrails are not negotiable. Better to exit gracefully with partial work than burn the budget.

 North Star Principles

Non-negotiable rules every pipeline follows. Violating these IS a bug.

#	Principle	Why
N1	Every interactable element is visited — text, dropdown, radio, checkbox, upload, toggle	Can't test what isn't mapped
N2	Styling / decorative elements are NEVER captured	Pollutes KB, wastes turns
N3	Last value on every field is a VALID / passable value before navigation	Gated wizards require valid state to advance
N4	LLM context per call stays low (< 5k tokens input where possible)	Prevents satisficing, loops, hallucination
N5	Locked / dependent elements → enter valid data on prerequisites to unlock them	Cascaded dropdowns, conditional UI

#	Principle	Why
N6	Toggles / conditional UI → new snapshot → capture revealed elements gracefully	Dynamic forms
N7	OCR-autofilled fields are LEFT ALONE unless a mandatory field is missing	App owns that data after validation
N8	File uploads wait for validation signal (OCR, "approved" indicator) before proceeding	Respect app's async verification
N9	Documents clearly named in <code>artifacts/test_files/{app}/</code> — LLM picks via token match	No hardcoding, flexible
N10	Pipeline 3 tests ONE element at a time and restores valid before moving on	Enables gated navigation
N11	Python orchestrates, LLM perceives	Determinism where possible, intelligence where needed
N12	Checkpoint after every successful action	Never lose work on crash or timeout
N13	Precision > Speed. A 10-min correct run beats a 2-min wrong one	Reruns cost more than careful first runs
N14	Every LLM claim is verified before entering KB	No silent acceptance of hallucinated data
N15	Every extracted element has a confidence score	Low confidence is visible, not hidden
N16	Atomic write after every element — not just at end	Crash never loses more than 1 element of work
N17	"Unsure" is a valid LLM response	Don't force false precision; route to human or retry

 Pipeline Responsibilities

Pipeline	Input	Output	Goal
1 EXTRACT	URL + app name + optional seed data	Layered KB (L0 / L1 / L2) with every element, selectors, options	Map the app completely. Unlock dependent sections intelligently. Never interfere with autofill.
2 PLAN	KB	Test cases (3 per inputtable element)	Pure LLM reasoning. Already works.

Pipeline	Input	Output	Goal
3 EXECUTE	KB + test cases	Test results with PASS / FAIL / BLOCKED + evidence	Run each test in isolation. Setup by replaying valid path. Test one field. Restore valid. Move on.

 Current State Snapshot (as of today)

What's actually working and what's partially working — so we always know where we stand.

Component	Status	Notes
Pipeline 1 (Path A — LLM-driven explore)	✅ Works	For simple single-page forms
Pipeline 1 (Path B — <code>GatedMultiSectionFlow</code>)	✅ Works	Page 2 style OCR-gated uploads. Proven on TECU page 2.
Pipeline 1 (Path B — <code>form_extract</code> hybrid)	✅ Works	Pages 3 + 4. Disabled-detection + retry on empty options.
Pipeline 2 (Plan)	✅ Works	Generates 40-50 cases, auto-injects upload cases.
Pipeline 3 (Execute — single screen)	🔄 Partial	Compound tools fine. Execute loop + prompt has bugs (see T0).
Pipeline 3 (Execute — multi-screen web)	❌ Broken	Nav not verified. LLM doesn't leave form valid. See Tier 0.
Mobile pipelines	✅ Works	Don't touch.
KB captured: TECU page 1	✅ Clean	9 elements
KB captured: TECU page 2 (3 sections)	✅ Clean	41 elements, real OCR data
KB captured: TECU page 3	✅ Clean	28 elements, 12/15 dropdowns with options, 3 dependent flagged
KB captured: TECU page 4	🔄 Partial	105 elements, but 17 element_id collisions
KB captured: TECU pages 5 + 6	❌ Not yet	Form_extract will handle when we get to them

 TIER 0 — Critical Blockers

Without these, Pipeline 3 cannot run end-to-end on TECU at all.

#	Wall	Pipeline	Action	Est.
0.1	Execute conflates setup-navigation with test-the-element	P3	Build ExecuteOrchestrator : Python drives setup (fills pages 1..N-1 with valid defaults), LLM runs 1 test on target element, Python restores valid, next test.	4-6h
0.2	Navigation between pages isn't verified (we trust a click that may not have advanced)	P1 + P3	navigate_to_screen : compare pre/post snapshots. If content barely changed → return False. Loop skips remaining screens cleanly.	30m
0.3	Element_id collisions from duplicate labels across sections (page 4: First Name x4)	P1	Section-aware element_ids: screen:section:label:type using nearest parent heading detected from DOM	45m
0.4	User-provided default data has no entry point	P1 + P3	--defaults CLI flag + defaults.yaml per-app: map field_name → valid_default . KB stores which values advance the form.	1h

Exit criterion: Pipeline 3 can test any page of TECU assuming prior pages have known-valid defaults. End-to-end on page 1 works cleanly.

TIER 1 — TECU Pages 1-4 Clean End-to-End

#	Wall	Pipeline	Action	Est.
1.1	Dependent dropdowns (Employer → Sector → Employment Type)	P1 + P3	KB stores depends_on . Execute chains: fill parent with KB default → snapshot → enumerate child → test child.	1h
1.2	N-repeating sub-forms (beneficiaries, insureds)	P1 + P3	Extract 1 template. In Execute: click "Add Another" N times, apply template per iteration with suffixed element_ids.	2h
1.3	LLM doesn't leave field valid at end of test	P3	Python enforcement, not prompt : after LLM test, Python re-fills target field with KB default before moving to next.	30m
1.4	OCR-gated uploads need replay during Execute setup	P3	Setup phase: if target page requires OCR'd uploads earlier, replay them deterministically using GatedMultiSectionFlow .	2h
1.5	OCR timing variance	P1 + P3	Already at 60s cap + late-success fallback. Add per-app override via config.	done
1.6	Conditional UI (toggles expand sections)	P1	Two-pass extract (baseline + all-yes) merged. Or single pass with user-pre-toggled state.	1.5h

#	Wall	Pipeline	Action	Est.
1.7	Auto-fill sections after OCR — agent mustn't interfere	P3	Skip rule: fields marked <code>behavior="auto-filled"</code> become <code>VERIFY_ONLY</code> , never tested.	30m
1.8	Block LLM-initiated page reloads / navigation during tests	P3	Remove <code>navigate_page</code> , <code>go_back</code> , <code>go_forward</code> from Execute tool list. Python controls navigation only.	15m

TIER 2 — Observability & Safety

#	Wall	Pipeline	Action	Est.
2.1	No live progress during long runs	All	Stream to <code>artifacts/results/{app}_live.json</code> ; dashboard polls.	1h
2.2	Failure loses context (no snapshot/screenshot)	P3	On FAIL / BLOCKED: capture snapshot + screenshot, link in report.	30m
2.3	No rerun-from-failure	P3	After Execute, report has <code>resume_from_case_id</code> . Next run skips earlier cases.	1h
2.4	Safe-run gate (refuse to run on prod URLs accidentally)	All	Refuse URLs containing <code>prod</code> / <code>production</code> unless <code>--i-know-what-im-doing</code> .	15m
2.5	Per-test cost attribution	P3	Budget tracker has totals; add per-case cost in report.	30m
2.6	Partial save on timeout for Pipeline 1	P1	Execute already does this. Mirror in <code>form_extract</code> / orchestrators.	45m
2.7	LLM hallucinations in sub-task outputs — every claim must be verified	P1 + P3	Chain-of-Verification (CoVe) on EVERY LLM claim entering the KB: dropdown options, element labels, required flags, dependencies, post-OCR fields, test-result observations. Each claim is independently verified against source snapshot before commit. Drop unsupported facts.	3h
2.8	Reflection: LLM reviews its own test result before recording	P3	After each Execute sub-task, self-check: "does my claim match the post-action snapshot?" Adds one cheap call per test.	1h

TIER 2.5 — Precision-First Patterns

Following from N13-N17: these make correctness structural, not optional. Each adds cost per run but **eliminates silent hallucinations** — the single biggest risk to KB quality.

#	Wall	Pipeline	Action	Est.
2.5a	Confidence scoring per L0 element	P1	Add <code>capture_confidence: float</code> field to L0Element. Every LLM-derived claim gets a score. < 0.7 = flagged for review. Manual/auto inspection of low-confidence entries.	1.5h
2.5b	Post-action verification on every interaction	P1 + P3	Every click, fill, upload is followed by a snapshot-verification step before moving on. MCP saying "success" is not enough — prove the DOM changed in the expected way.	2h
2.5c	Explicit "unsure" route for LLM sub-tasks	P1 + P3	Sub-task schemas include <code>{status: "sure" \ "unsure", reason: str}</code> . On "unsure", Python retries with different prompt framing, or marks element for human review. No forced hallucination.	2h
2.5d	Multi-pass verification on critical KB data	P1	Dropdown options and dependency chains extracted twice via different prompts. Agreement → commit. Disagreement → third pass + flag.	2h
2.5e	Anomaly detection across extracted screens	P1	After extraction, statistical sanity check: "14/15 dropdowns have options, this one has 0 — suspicious". Auto-flag outliers for verification.	1h
2.5f	Atomic per-element KB writes	P1	After each element successfully extracted + verified, immediately persist. Crash after N elements = resume from N+1, not restart.	1.5h
2.5g	Per-element capture log (audit trail)	P1	Every element records: (raw_llm_response, verified_claim, confidence, elapsed_ms, retries). Stored alongside L2. Debugging gold when something looks wrong.	1h
2.5h	Guardrail enforcement — every orchestrator honors hard caps	All	<code>GuardrailContext</code> wraps every sub-task: tracks calls / time / cost / retries, raises <code>GuardrailExit</code> on cap hit. Orchestrator catches, saves partial, continues. See Operational Guardrails table at top of plan.	2h

Exit criterion: a clean page 3 re-extraction shows all 15 dropdowns with `capture_confidence` ≥ 0.85 , zero silent drops, full per-element audit log available.

Cost impact: ~3× LLM spend per extraction. We ship for correctness, not per-run cost. A 10-minute, \$0.40 extraction that's right is INFINITELY better than a 2-minute, \$0.05 extraction we have to redo.

TIER 3 — Pages 5-6 + Widget Generalization

#	Wall	Pipeline	Action	Est.
3.1	TECU page 5 (PEP/FATCA) not extracted	P1	Run <code>form_extract</code> with same tool. No code change expected.	10m
3.2	TECU page 6 (PDF/Other Details) not extracted	P1	Same as 3.1.	10m
3.3	Custom widget libraries (react-select, Ant Design, Quill, etc.)	P1	Library detection heuristic in <code>form_extract</code> ; per-library open/close tactics.	3h
3.4	Autocomplete / typeahead inputs	P3	Compound tool: type 2-3 chars → wait → pick from dropdown.	1h
3.5	Date range pickers (two linked fields)	P3	Specialized tool detecting pair patterns.	1.5h
3.6	Rich text editors (Quill, TinyMCE)	P3	Direct <code>.innerHTML</code> write + change event dispatch.	1h
3.7	Drag-drop file uploads (no <code>input[type=file]</code>)	P3	Synthesize <code>DataTransfer</code> + drop event via JS.	1.5h

TIER 4 — Session / Infrastructure Hardening

#	Wall	Pipeline	Action	Est.
4.1	401 / session expiry mid-run	All	Detect auth-failure page shape, pause with manual-resume, continue.	1.5h
4.2	Chrome profile corruption	All	Auto-detect + reset profile on launch failure.	30m
4.3	CAPTCHA on real submission	P3	Detect + pause with manual-solve + resume.	1h
4.4	2FA during login flows	All	Pre-run manual login via <code>--wait</code> + save session cookie, OR integrate TOTP.	1h
4.5	Rate limiting / backend throttling	P3	Exponential backoff + resume from checkpoint.	1h
4.6	Credentials management for test runs	All	<code>.env</code> per app with test credentials; never committed.	30m

TIER 5 — Future Edge Cases

#	Wall	Pipeline	Approach
5.1	Shadow DOM elements	P1	JS pierce via <code>element.shadowRoot</code> recursion
5.2	Iframes	P1	Switch frame context or <code>contentDocument</code> traversal

#	Wall	Pipeline	Approach
5.3	Maps / location pickers	P3	Manual mode: mark as SKIP with reason
5.4	A/B tested UIs (selector flipping)	P3	Multi-locator fallback — L1 already supports, needs strengthening
5.5	Multi-language apps	All	Set Accept-Language header + English-forced prompts
5.6	Real-money / transactional flows	P3	Test-env URL allowlist, refuse prod patterns

⚙️ TIER 6 — Model / Infra Limits

#	Wall	Status	Fix
6.1	Reasoning model satisficing (gpt-5.1)	⚠️ Known	Path B orchestrator — Python orchestrates, LLM narrates
6.2	Non-reasoning model looping (gpt-5)	⚠️ Known	Loop detection + force output (already in engine)
6.3	API 120s timeout per turn	🔄 Partial	Already caught in execute. Mirror to explore + orchestrators.
6.4	Quadratic context growth in single-agent runs	✅ Solved for P1	Path B. Needs same pattern in P3 (Tier 0.1).
6.5	MCP tool timeouts / slow responses	🔄 Partial	Warn at 5s already. Add retry with backoff on confirmed timeouts.

🏗️ Build Order — Most Efficient Path

Each step unlocks the next. Don't skip ahead.

FOUNDATIONAL (~7 hr)

Step 1

[T0.2]

Nav verification via snapshot diff

30m

Step 2

[T0.3]

Section-aware element_ids

45m

Step 3

[T0.4]

--defaults channel + defaults.yaml

1h

Step 4

[T2.1]

Live progress stream

1h

Step 5

[T2.2]

Snapshot/screenshot on FAIL/BLOCKED

30m

Step 6

[T1.8]

Block LLM navigation during tests

15m

Step 7

Re-extract page 4 (clean 105 unique IDs)

30m

⇒ Foundation done. Ready to build ExecuteOrchestrator.

PIPELINE 3 CORE (~8 hr)

Step 8

[T0.1]

ExecuteOrchestrator (setup/test split)

5h

Step 9

[T1.3]

Python enforces valid state post-test

30m

Step 10	[T1.4]	OCR-gated upload replay in setup	2h
Step 11	[T1.7]	Auto-fill VERIFY_ONLY rules	30m
⇒ TECU pages 1 + 2 Execute end-to-end works.			
TECU COMPLETE (~4 hr)			
Step 12	[T1.1]	Dependent dropdown chaining	1h
Step 13	[T1.2]	N-repeating sub-forms	2h
Step 14	[T3.1]	Extract page 5	10m
Step 15	[T3.2]	Extract page 6	10m
Step 16	[T1.6]	Conditional UI two-pass extract	1.5h
⇒ TECU 1-6 end-to-end. 🎉 Real bugs captured. ~19 hr total			

Everything beyond is generalization and polish.

✅ Success Metrics Per Tier

How we know we're actually done, not just "works on my machine".

Tier	Success Metric
T0	<code>python -m qa.cli execute TECU</code> on page 1 alone → 90%+ tests PASS or legitimately FAIL (no BLOCKED on infra).
T1	Full TECU run: pages 1-4, all Execute tests complete, results have real signal (not all SKIP).
T2	During a 20-min Execute run, dashboard shows live element-by-element progress. A crash loses < 1 test.
T3	Page 5 + 6 captured. Full 6-page TECU run completes with real bug report.
T4	Session-expiry mid-run recovers without losing prior results.
T5	Re-run on a second app (demoqa or real KYC clone) with zero code changes, works end-to-end.

⚠️ Risk Register

Things that could derail Week 1. Named so they're less scary.

Risk	Mitigation
TECU backend goes flaky (401s, rate limits)	Fall back to captured KB + play recordings. Revisit when back.
ExecuteOrchestrator takes 10h instead of 5h	Ship narrower scope first: single-screen orchestrator, multi-screen next day.

Risk	Mitigation
Hidden widget library we haven't seen	Log + dump DOM for inspection. Manual fallback: user pre-operates before extract.
Element IDs still collide after T0.3	Add UUID suffix fallback for still-duplicated names.
gpt-5.1 still satisfices even in narrow sub-tasks	Switch that sub-task to gpt-5 (non-reasoning) or haiku.
Chrome MCP breaks/updates	Version pin in adapter. Revert if breaking change appears.

 AI Patterns In Use

Explicit mapping of known patterns to where they live in our architecture. Useful for future contributors and for defending design choices.

Pattern	Where Used	Status
Orchestrator-Worker	Path B orchestrators (GatedMultiSectionFlow , form_extract). Python orchestrates, LLM sub-tasks work.	 Implemented
Model Context Protocol (MCP)	Chrome DevTools MCP (web) + mobile-mcp (mobile)	 Implemented
Retrieval-Augmented Generation	KB L0 index — we fetch only the target screen's elements per LLM call; irrelevant context stays out	 Implemented
Plan-and-Solve	Pipeline 2 (Plan) → Pipeline 3 (Execute) separation	 Implemented
ReAct	Single-LLM explore (Path A). Used only where Path B doesn't fit.	 Implemented
Agent-Computer Interface (ACI)	PlatformAdapter protocol — standardized interface across web / mobile backends	 Implemented
Chain-of-Verification (CoVe)	Planned: wrap key LLM sub-tasks (options extraction, state classification, test-result claims). See T2.7.	 To build
Reflection	Planned: LLM reviews its own test result vs observed snapshot before recording. See T2.8.	 To build

Patterns evaluated and rejected for our problem (see [docs/ai-patterns.pdf](#)): Tree/Graph of Thought, LATs, Deep Research Agents, Toolformer, Multi-Agent Debate, Self-RAG, Generative Agents, Meta-Prompting, Skeleton of Thought. Overkill or wrong fit for linear extract-plan-execute flows.

 Operating Rules

- 1. **One wall at a time, in priority order.** No jumping.

2. **Every new wall gets added to this list.** Never forget.
3. **Commit after every wall solved.** Never lose work.
4. **Hardcode if needed today; elegance later.** Ship > polish.
5. **The agent is narrow** — Python orchestrates, LLM perceives.
6. **No pivoting.** TECU is the benchmark. Other apps come after TECU works.
7. **Extract spec, explicit additions:**
 - Skip `role="presentation", aria-hidden="true", aria-disabled="true"` non-interactive elements.
 - Capture validation rules visible in DOM (`maxlength, pattern, type="email"`).
 - Record `depends_on` when field B is disabled until A is filled.
 - Record `behavior: "auto-filled"` for fields populated by OCR/API.
 - Detect repeating templates when "Add Another" visible; mark sub-section as template.
 - Screenshot per page as an artifact for visual verification.
 - Per-element locator confidence: CSS > data-testid > XPath > uid > coordinates.

Tomorrow's First Move

Step 1: Solve T0.2 (nav verification) — 30 minutes, highest ROI for smallest effort. Unblocks tonight's Execute crash immediately.

Then proceed in order through the Build Order above.

17 Change Log

- **Day 2 PM (today):** Initial plan written. 6 tiers, 35+ walls documented. Current state snapshot added.
- **Day 2 PM (today):** Added CoVe (T2.7) and Reflection (T2.8). Added AI Patterns In Use section mapping architecture to named patterns.
- **Day 2 PM (today): Philosophy shift — precision over speed.** Added Design Philosophy section up top. Added principles N13-N17. New Tier 2.5 (Precision-First Patterns) with 7 walls: confidence scoring, post-action verification, "unsure" routing, multi-pass verification, anomaly detection, atomic per-element writes, capture audit log. CoVe scope upgraded from "key sub-tasks" to "every LLM claim entering KB".
- **Day 2 PM (today): Bounded precision.** Replaced "however long / as many calls as needed" with explicit Operational Guardrails table (per-element / per-page / per-test / per-run soft + hard caps). Added Graceful Exit Contract. Added T2.5h Guardrail enforcement wall.
- **Day 2 PM (today): Recalibrated guardrails to actual measured costs.** Earlier proposal of \$2/page was 30× inflation. Tightened to \$0.30 hard cap per page, \$1.50 per full pipeline run. Added "How We Stay Cheap While Adding Precision" section with 8 engineering techniques (prefix caching, compound tools, haiku verify, batch verify, structural pre-checks, etc.). Phase 1 now enforces tight caps from day one via better engineering, not looser budgets.
- **[future entries added here as we progress]**